

Alerting & Grafana Dashboard

Sesi ini menutup Week 15 dengan membangun lapisan terakhir dari observability stack: visualisasi metric lewat Grafana dan sistem alerting otomatis lewat Prometheus Alertmanager. Setelah sesi ini, sistem kamu tidak hanya bisa diamati — ia bisa berteriak sendiri ketika ada yang salah.

1. Arsitektur Observability Stack

Stack observability yang lengkap terdiri dari tiga komponen yang bekerja bersama. **Analogi:** bayangkan pabrik dengan sensor di setiap mesin. Sensor-sensor itu merekam angka ke buku log besar (Prometheus). Layar monitor di control room menampilkan grafik dari semua sensor secara real-time (Grafana). Dan alarm otomatis berbunyi ketika sensor melewati batas tertentu (Alertmanager).

Komponen	Peran	Analogi
Prometheus	Scrape & simpan time-series metrics dari target secara periodik	Buku log setiap pabrik
Grafana	Visualisasikan metric dari Prometheus via PromQL query	Layar di dashboard yang bisa dibaca manusia
Alertmanager	Terima alert dari Prometheus, lakukan deduplication & grouping	Sistem alerting berbasis event (Slack, email, dll)

Alur data: FastAPI expose /metrics → Prometheus scrape setiap 15s → Grafana query Prometheus via PromQL → Jika alert rule FIRING, Prometheus push ke Alertmanager → Alertmanager routing notifikasi ke Slack/email.

Penting tentang networking di Docker: Grafana dan Prometheus berjalan dalam container terpisah. Dari dalam container Grafana, *localhost* berarti container Grafana itu sendiri — bukan host machine. Untuk reach Prometheus, gunakan nama service Docker: **http://prometheus:9090**, bukan http://localhost:9090. Semua container dalam satu Docker network bisa saling reach via nama service.

2. PromQL — Query Language Prometheus

PromQL adalah bahasa query untuk data time-series. Berbeda dari SQL yang query data tabular, PromQL query data yang berubah seiring waktu. Kalau SQL tanya 'berapa total penjualan?', PromQL tanya 'berapa rata-rata request per detik dalam 5 menit terakhir?'

Tiga Tipe Data

Tipe	Penjelasan	Contoh
Instant vector	Nilai metric pada satu titik waktu — hasil query instan	http_requests_total
Range vector	Kumpulan nilai dalam rentang waktu. Ditandai [duration]. <code>http_requests_total[5m]</code> seperti <code>rate()</code>	
Scalar	Angka tunggal tanpa label	42, 0.05

Fungsi-Fungsi Penting

Fungsi	Kegunaan	Contoh Query
rate()	Rata-rata per detik dari counter dalam window. Paling sering dipakai untuk <code>rate(http_requests_total[5m])</code> est rate.	
irate()	Seperti <code>rate()</code> tapi hanya 2 sample terakhir — lebih responsive (<code>irate(http_requests_total[5m])</code>)	
increase()	Total kenaikan counter dalam window. Berguna untuk "berapa banyak data bertambah" (<code>increase(http_requests_total[1h])</code>)	
sum() by()	Aggregate semua time-series, dikelompokkan berdasarkan label (<code>sum(rate(...)) by (method)</code>)	
histogram_quantile()	Hitung persentil (P50/P95/P99) dari histogram. Sangat berguna untuk <code>histogram_quantile(0.95, rate(...bucket[5m]))</code>	
Label filter {}	Filter berdasarkan label. Operator: <code>= != =~ (regex) !~</code>	<code>http_requests_total{status_code=~"5.."} </code>

Penting — rate() vs counter kumulatif: Counter seperti `http_request_total` hanya naik dan tidak pernah turun. Angka 654 berarti 'sejak server dihidupkan, ada 654 request total'. `rate()` tidak peduli total kumulatif itu — ia hanya mengukur *seberapa cepat counter naik dalam window waktu tertentu*. Ketika traffic berhenti, counter tidak bertambah → `rate()` kembali ke 0. Ini adalah perbedaan fundamental antara 'sudah berapa banyak' vs 'seberapa cepat sekarang'.

Pola ratio query (paling penting untuk error rate): Error rate = jumlah error dibagi total request. Dalam PromQL:

```
sum(rate(http_request_duration_seconds_count{status_code=~"4..|5.."}[5m]))
/
sum(rate(http_request_duration_seconds_count[5m]))
* 100
```

3. Alert Rules & State Machine

Analogi: Prometheus evaluasi alert rules seperti satpam yang setiap 15 detik mengecek papan sensor. Kalau sensor menunjukkan angka buruk, satpam mencatat kondisi itu sebagai PENDING. Kalau kondisi buruk bertahan selama minimal *for duration*, barulah satpam mengirim laporan ke Alertmanager sebagai FIRING. Ini mencegah alarm berbunyi hanya karena lonjakan sesaat selama 2 detik.

State Machine Alert

State	Kondisi	Warna di Prometheus UI
INACTIVE	expr bernilai false — kondisi normal	Hijau
PENDING	expr bernilai true, tapi belum melewati for duration	Kuning/Orange
FIRING	expr true DAN sudah melewati for duration — alert dikirim ke Alertmanager	Merah
RESOLVED	expr kembali false setelah sebelumnya FIRING	Hijau (dengan notifikasi resolved)

Struktur alert_rules.yml

```
groups:
  - name: api_alerts
    rules:
      - alert: HighErrorRate
        expr: |
          sum(rate(http_request_duration_seconds_count{status_code=~"4..|5.."}[1m]))
            > 0.05
        for: 1m           # harus bertahan 1 menit sebelum FIRING
        labels:
          severity: critical
        annotations:
          summary: "Lonjakan error terdeteksi"
          description: "Rate error: {{ $value }}"
```

Komponen penting dalam alert rule: **expr** adalah PromQL yang dievaluasi — hasilnya harus truthy (> 0) untuk masuk PENDING. **for** adalah durasi minimum sebelum FIRING — mencegah false positive dari spike sesaat. **\$value** dalam annotations adalah nilai dari expr saat alert FIRING. **\$labels.xxx** mengakses label dari metric yang trigger alert.

YAML gotcha — karakter khusus: Tanda ~ dalam PromQL (untuk regex match seperti `status_code=~"4..|5.."`) bisa menyebabkan YAML parse error jika tidak di-handle dengan benar. Gunakan block scalar (|) untuk expr multi-baris, atau pastikan indentasi menggunakan spaces bukan tabs. Satu karakter tab saja akan membuat Prometheus crash saat load config.

4. Alertmanager — Routing Notifikasi

Alertmanager menerima alert FIRING dari Prometheus, lalu memutuskan: kapan kirim, ke mana kirim, dan seberapa sering. Tiga fungsi utamanya: **Grouping** (kumpulkan alert serupa jadi satu

notifikasi), **Deduplication** (jangan kirim alert yang sama berkali-kali), **Silencing** (matikan alert tertentu saat maintenance window).

```
route:
  receiver: 'default-receiver'
  group_by: ['alertname', 'severity']
  group_wait: 30s      # tunggu 30s kumpulkan burst alert
  group_interval: 5m    # interval minimum kirim ke group yang sama
  repeat_interval: 1h   # kirim ulang jika masih FIRING setelah 1 jam

  routes:
    - match:
        severity: critical
      receiver: 'critical-receiver'
      repeat_interval: 15m  # critical: repeat lebih sering

receivers:
  - name: 'default-receiver'
    webhook_configs:
      - url: 'http://webhook-logger:5001/alert'
        send_resolved: true
```

5. Grafana Dashboard

Grafana memvisualisasikan data dari Prometheus via PromQL query. Setiap panel adalah satu visualisasi dengan satu atau lebih query.

Tipe Panel Utama

Type Panel	Kegunaan	Contoh Metric
Time series	Grafik garis untuk metric yang berubah seiring waktu	Request rate, latency trend
Stat	Angka besar tunggal untuk current value	Total requests, error rate saat ini
Gauge	Dial/lingkaran untuk nilai dalam range	CPU usage, active requests
Table	Data multi-label dalam format tabel	Per-endpoint breakdown

4 Panel Esensial untuk ML API Dashboard

Panel	Query PromQL	Unit
Request Rate per Endpoint	sum(rate(http_request_duration_seconds_count[5m])) by (path)	req/s
Error Rate %	sum(rate(...{status_code=~"4.. 5.."}[5m])) / sum(rate(...[5m])) * 100	Percent (0-100)
Latency P50/P95/P99	histogram_quantile(0.95, sum(rate(...bucket[5m])) by (le))	seconds (s)
Active Requests	http_server_active_requests atau gauge metric yang tersedia	short

Provisioning vs konfigurasi manual: Grafana bisa dikonfigurasi via file YAML (provisioning) agar reproducible dan tidak hilang saat container restart. Datasource dan dashboard bisa di-define di file,

lalu Grafana load otomatis saat startup. Selalu export dashboard sebagai JSON dan simpan ke repository — ini production practice.

Grafana bukan hanya untuk request traffic: Grafana menampilkan apapun yang di-expose sebagai metric. Untuk ML system, kamu bisa expose: akurasi model (Gauge), distribusi confidence score (Histogram), jumlah data drift terdeteksi (Counter), waktu inferensi (Histogram). Batasan Grafana adalah apa yang kamu instrumentasi dari kode — bukan hanya HTTP traffic.

Siapa yang pakai Grafana: Developer/ML Engineer untuk dashboard teknis (latency, error rate), Admin/DevOps untuk dashboard infrastruktur (CPU, memory, container health), Business/PM untuk dashboard bisnis (prediksi per hari, success rate dalam bahasa bisnis). User aplikasi tidak pernah menyentuh Grafana — mereka hanya merasakan manfaatnya karena masalah diselesaikan sebelum mereka sadar ada yang salah.

6. Pelajaran dari Debugging Hari Ini

Beberapa hal penting yang muncul dari praktik langsung hari ini:

- **YAML tidak toleran terhadap tab.** Satu karakter tab dalam `alert_rules.yml` cukup untuk membuat Prometheus crash dan menolak start. Selalu gunakan spaces. Jika edit file config, verifikasi syntax sebelum restart container.
- **`rate()` = 0 ketika traffic berhenti.** Counter kumulatif (654 requests) tidak berarti rate tinggi. `rate()` mengukur kecepatan perubahan — bukan total. Traffic generator harus terus berjalan agar `rate()` menunjukkan nilai non-zero.
- **localhost di dalam container bukan host machine.** Grafana di container tidak bisa reach Prometheus via `localhost:9090`. Gunakan nama service Docker (`prometheus:9090`) untuk komunikasi antar container dalam satu network.
- **422 vs 500 adalah perbedaan alert yang tidak trigger.** FastAPI return 422 untuk Pydantic validation error, bukan 500. Alert rule yang filter `status_code='500'` tidak akan trigger dari validation error. Gunakan regex `status_code=~'4..|5..'` untuk cover semua error.
- **for: duration mencegah false positive.** Alert tidak langsung FIRING begitu `expr` true. Harus bertahan selama `for duration` (misal 1 menit). Ini mencegah spike sesaat yang normal menyebabkan banjir notifikasi.

7. Output Files Hari Ini

File	Deskripsi
<code>prometheus/alert_rules.yml</code>	Alert rules: <code>ApiInstanceDown</code> , <code>HighErrorRate</code> , <code>SlowMachineLearningComputation</code>
<code>alertmanager/alertmanager.yml</code>	Routing config: <code>webhook receiver</code> , <code>group_wait</code> , <code>repeat_interval</code>
<code>grafana/provisioning/datasources/prometheus.yml</code>	Datasource provisioning — connect Grafana ke Prometheus

File	Deskripsi
grafana/provisioning/dashboards/api_dashboard.json	Dashboard JSON export — 3 panel: request rate, error rate, latency
docker-compose.yml	Orchestrasi: api, prometheus, grafana, alertmanager, jaeger, postgres